Advanced Traffic Demand Generation in SUMO: ML-based Prediction of Flow Rate based on Real-world Measured Datasets

Anjie Qiu, Prapul Alemada Sathish, Donglin Wang and Hans D. Schotten
Chair for Wireless Communication & Navigation
RPTU Kaiserslautern-Landau
{aqiu, alemada, dwang, schotten}@rptu.de

***Abstract*—Realistic vehicle traffic simulations have been a subject of great interest in many fields, particularly in the Intelligent Transport Systems (ITS), as they can help improve transportation efficiency, safety, and sustainability. Simulation of Urban MObility (SUMO) is a powerful tool for vehicular traffic simulations; however, traditional methods of generating traffic demand are usually based on random trips or the use of real-world datasets, which can be difficult to obtain. More advanced approaches create algorithms to generate trips based on multimodal activities. However, these approaches require either a large amount of computational resources or a great deal of effort in deploying measurement hardware. Therefore, we proposed a new scheme to generate large-scale traffic demands, where a real-world dataset is fed into an ML model to predict traffic flows. We then calibrate the traffic flows to match the predicted ones, resulting in realistic traffic flows for urban scenarios. The approach is highly computationally efficient and can be applied to any city scenario. Furthermore, the training data set and the ML model can be updated for more precise predictions. In the end, we create the whole map of Berlin and generate around 1.2 million vehicles as a 24-hour trial, and compared the predicted model with the real-world measured dataset. As a result, more than 85 percent of the monitored lanes show a mismatch of less than 5 vehicles in 24 hours, and a general traffic pattern of Berlin is revealed and matched.**

***Keywords*—Vehicular-to-Everything, Machine Learning, Urban Mobility, Vehicle Traffic Generation.**

I. Introduction

SUMO is an increasingly popular open source traffic simulation software for modeling vehicular traffic on road networks that models individual vehicles and real-time interactions with road obstacles (like traffic lights, gas emissions, fuel consumption) in real time. In addition, SUMO is an open source traffic simulator. With it, various vehicle traffic scenarios can be simulated. One of the most relevant applications of SUMO is in the ITS area, where it can provide intelligent mobility solutions to improve transportation efficiency and sustainability. With the support of SUMO for realistic driver behavior and emission modeling, the environmental impact of different transportation systems can be evaluated and also used to optimize traffic management strategies, develop intelligent routing algorithms, etc.

Nowadays, cities are increasingly looking for smart mobility solutions that are tailored to their needs. To do this, it is necessary to create a realistic traffic pattern in the city. This is also beneficial for networking technologies, as it allows them to simulate vehicular networks. Connected vehicles are a key part of the telecommunications revolution, known as Vehicle-to-Anything (V2X). Realistic mobility scenarios that represent a city are essential to analyze traffic patterns at different times, such as peak and off-peak hours. They are also useful for identifying and analyzing areas of traffic congestion in the road network, such as when multiple lanes merge into one.

The generation of urban mobility scenarios is advantageous for a number of applications. However, a major challenge in this context is the lack of availability of publicly available scenarios, especially for large cities. Some of the large-scale mobility scenarios that exist make use of proprietary data. The creation of a realistic mobility scenario is also a challenging exercise that involves numerous configuration files, substantial amounts of top-quality data, specialized expertise, and manual adjustments. Although SUMO provides certain tools that simplify scenario generation without requiring many prerequisites, these tools are inadequate for generating mobility scenarios for large cities due to exponentially increased computational complexity. This paper seeks to tackle the challenge of quickly and efficiently generating mobility scenarios for large cities. To do this, a Machine Learning (ML)-based prediction method is used to create realistic vehicular mobility flows. To demonstrate the effectiveness of the method, it is implemented in Berlin, a large metropolitan area with more than one million population.

The following sections constitute the remainder of this paper. In Chapter II the various classes of vehicular traffic simulation ajpproaches are discussed. In Chapter III, the methodology of generating realistic traffic stimulation is introduced. Then, Chapter IV introduces an approach that involves machine learning methods to make predictions about traffic flow, and also how mobility generation of Berlin is made based on the predicted traffic flows. The results and their evaluation are discussed in Chapter V, while Chapter VI concludes this work and mentions various improvements that could be incorporated to improve the results.

II. Related Work

Existing approaches to the generation of urban mobility include the Bologna scenario [1], the Austria-West scenario

979-8-3503-8741-4/24/$31.00 ©2024 IEEE

TABLE I: Comparison of existing traffic scenarios

Scenario	Bologna	Austria	Turin	Cologne	Monaco	Luxembourg
Area	20.6sq.km	600sq.km	11982sq.km	400sq.km	73.64sq.km	155.9sq.km
Vehicles	10,333	1.6 million	2 million	1.2 million	3,500	14,000
Monitors	636	245,000	-	-	-	-
Duration	1hour	24hours	24hours	24hours	morning rushing-hour	peak and off-peak hours
Data Source	real	real	real	travel pattern based	activity based	activitygen
Data Nature	proprietary	proprietary	proprietary	-	-	-
Availability	✓	×	✓	✓	✓	✓
Multi-modality	×	×	×	×	✓	✓

[2], the Turin scenario [3], the Cologne scenario [4] , the Monaco scenario [5], and the Luxembourg scenario [6]. The Bologna scenario uses traffic datasets from induction loops spread throughout the city and Origin Destination (OD) matrices to generate a 1 hour period of traffic. The Austria West scenario uses data from sources such as Vehicle Data Loop (VDL) and Floating Car Data (FCD) from the local traffic department and extrapolation methods to estimate traffic values for roads not covered by sensors. The Turin scenario uses OD matrices from the transport department and traffic signal logic data to generate traffic flow. The Cologne scenario uses a synthetic population to create an OD matrix that replicates the daily movements of the inhabitants of the area. The Monaco scenario creates a synthetic traffic demand using statistical information on the population and a simplified mobility model. The Luxembourg scenario uses the ***activitygen*** tool in SUMO to generate traffic demand and offers a multimodal mobility generation with public transport, motorbikes, bikes, and pedestrians. All the mobility scenarios mentioned in this section have their own advantages and disadvantages. Some of the important features of the scenarios have been mentioned in Table I.

Also, in our previous works [7] and [8], where traffic demands for different German cities are generated for evaluating the impact of Connected Cooperative Automated Mobility (CCAM) on the Green House Gas (GHG) emissions. However, we failed to generate a large-scale traffic demands using the above-mentioned methods for metropolis such as city of Berlin and Munich and instead, random trips are deployed in the simulation.

III. Methodology

Generation of a scenario from scratch is a time-consuming process. Existing approaches rely on either available real detector datasets or synthetic models based on activity patterns. [1], [3], [9] use real traffic data that produce accurate results but have low generalization and depend on the traffic data provider. For the generation of a mobility scenario of a particular city, one needs to deal with the problem of data availability and configuration steps. In addition, traffic detector data are sometimes proprietary and difficult to obtain. The generation of scenarios for cities in [4], [10], [6], [5] uses tools that make use of activity-based mobility modeling. The tool activitygen is used, but precision relies on a detailed statistics file created by the user, which can be long and cumbersome. When the mobility scenario of a particular city is required, there are few options available. Sumo Activity Generation (SAGA) offers a solution, but fails when the scenario of a large city is required.

Thus, we propose a new ML-based approach to generate traffic demands for all traffic networks. First, we feed the ML-model with existing real-world traffic data, so it can learn the traffic pattern and adapt to city characters. After training, an arbitrary city network can be fed into the model, and traffic flows for the lanes can be predicted and calibrated into the traffic routes in SUMO using the tool "routeSampler.py". This produces a realistic traffic pattern for any city network, following the ML-predicted traffic patterns, with faster speed and efficient resource requirements.

A. The Training Dataset: real-world UTD19 Dataset

To develop a prediction model, it is essential to have a large and representative dataset. For the prediction model, UTD19 [11] may be useful, as it consists of extensive traffic data collected from more than ***20,000 stationary detectors*** located on urban roads in ***40 cities*** throughout the world. Figure 1 shows the geographic location of all detectors contained in the entire data set. Most of them are in cities of the European continent.

This is one of the largest collections of real-world traffic data from cities around the world currently available to the public. Data are collected by the ETH Zurich Institute for Transport Planning Systems during a research campaign that took place between 2017 and 2019. Most of the data is collected from loop detectors that capture vehicle flow in small intervals, typically every 3 to 5 minutes. The dataset also includes geospatial information on detector locations, along with Open Street Map (OSM) attributes such as lat-long coordinates geocoded in the World Geodetic System 1984 (WGS84)(World Geodetic System 1984) coordinate system. The Table II illustrates the features of the original detectors of the UTD19 dataset. This table provides an overview of the features of detectors in Augsburg, Germany.

Not all columns in the dataset will be useful for the training stage. For example, the "lat" and "long" features that express the latitude and longitude of the detector are only useful for identifying the location of a detector and have no direct relation to the measured traffic flow value. Therefore, these features will be removed before using the dataset for training. Additionally, features that could improve the accuracy of the

detid	length	pos	fclass	road	limit	citycode	lanes	linkid	long	lat
U1-52G	0.196037	0.005512	secondary	Gögginger Straße	50	augsburg	1.0	72.0	10.889553	48.359957
U1-51G	0.130039	0.004013	secondary	Gögginger Straße	50	augsburg	1.0	73.0	10.889601	48.359945
U1-62	0.065183	0.024465	secondary	Rosenaustraße	50	augsburg	1.0	68.0	10.889361	48.360578
U1-61	0.436384	0.021137	secondary	Rosenaustraße	50	augsburg	1.0	67.0	10.889344	48.360556
U1-42	0.099691	0.039163	tertiary	Gögginger Straße	50	augsburg	1.0	0.0	10.890404	48.360719

TABLE II: Details of some of the detectors of the UTD-19 dataset with the explanation of the columns: **detid** = detector ID; **length** = length of the monitored lane in km; **pos** = distance between the detector and the next major intersection in km; **fclass** = classification of the road from OSM; **road** = name of the road; **limit** = speed limit; **citycode** = name of the city; **lanes** = number of lanes monitored; **linkid** = id number linking detectors and lanes; **long** = longitude; **lat** = latitude



Fig. 1: Locations of the measurement campaigns in the UTD-19 dataset

prediction can be added. According to [11], the density of the road network or the kilometers of lanes measured per surface area is an important factor that determines the traffic capacity of a network. Other characteristics, such as the population or population density of a city, could also be important in explaining differences in traffic from city to city. Thus, the data set calculates and combines the density of the lane or the total length of the lanes per surface area of cities. These data were extracted manually from the OSM file for each respective city. Furthermore, features such as population and population density of cities are extracted and added to the data set to check its impact on the prediction accuracy.

B. The baseline: real-world Berlin dataset

To evaluate the performance of the predicted traffic flows of the ML-model, a test (verification) dataset (not included in the training dataset), and therefore, the Berlin traffic dataset [12] is chosen for verification. The dataset is provided by the Verkehrsinformationszentrale Berlin (Traffic Information Center Berlin), where over 240 traffic detectors are deployed across the city of Berlin to measure the number, composition (cars, trucks) and speed of vehicles on streets. The dataset provides monthly archived data per location on an hourly basis.

With the help of these traffic datasets of Berlin, a comparison can be made between these baseline measured data to the predicted traffic flows using our proposed methodology.

A fragment of the dataset is presented in Table III.

IV. Workflow of the ML-based Traffic Generation

With a total population of more than 3 million and a total area of 892 square kilometers, Berlin belongs to the class of metropolis cities which are large cities with a high population. Since real traffic count readings are available for the city of Berlin as a test dataset, the proposed prediction approach based on ML for the generation of scenarios is applied to the city of Berlin and the results are compared with the actual data.

A. Step1: Get Predictions from the Trained Model

The geo-coordinates of the existing traffic count detectors of the city of Berlin are available at [12] in Excel format. In addition to the coordinate details, other specific details such as the length of the lane and the number of lanes are also available from OSM. It is loaded in Jupyter Notebook [13] as Pandas Dataframe [14] for easy processing of the data. After preprocessing the UTD-19 dataset mentioned in previous Section, the whole dataset is about to be fed into the ML-based model, and here we used the different prediction model, such as Linerar Regressor, linear Support Vector Machine (SVM) Regressor and ***Random Forest Regressor*** [15].

However, before the training process, we also merge the preprocessed UTD-19 dataset with the test Berlin dataset, for example, the measurement time interval, lane names, etc. In addition to that, we also add additional features to the training dataset, such as population, population density, etc. Finally, the fully preprocessed dataset with features and target is shown in Table IV.

B. Step2: Create the Network of the Scenario

The process of network building which involves creating a digital representation of the road network is an elementary step in the scenario generation process. The ***netconvert*** from

detid	date	hour	quality	N_{veh}	v_{veh}	$N_{passenger}$	$v_{passenger}$	N_{truck}	v_{truck}
100101010000167	01.06.2023	0	1.0	136	73	102	74	34	72
100101010000167	01.06.2023	1	1.0	120	74	85	74	35	73
100101010000167	01.06.2023	2	1.0	135	70	84	70	51	70

TABLE III: Details of one detector in the Berlin dataset with column explanation: **detid** = detector ID; **date** = measuring date; **hour** = hourly measurement interval, 0 stands for 0:00 to 0:59; **quality** = 1 for a valid and 0 for a malicious measurement; N_{veh} = number of all vehicles counted; v_{veh} = average speed of counted vehicles in km/h; $N_{passenger}$ = number of passenger cars; $v_{passenger}$ = average speed of counted passenger cars in km/h; N_{truck} = number of trucks; v_{truck} = average speed of counted trucks in km/h;

Time [hh:mm]	00:00	01:00	02:00	03:00	04:00	05:00
Flow [veh/h]	**143**	**83**	**85**	**69**	**177**	**451**

TABLE IV: A fragment of the predicted dataset of traffic flows (last column) from the pre-processed UTD-19 dataset

SUMO is a convenient tool as it can be used to generate a network file from a variety of input files, the most popular source being OSM files [16], as it is freely available and can be easily accessed and downloaded. OSM is a collaborative project to create a free and open map of the world, created and maintained by volunteers around the world. OSM data includes information about roads, highways, and other transportation infrastructure, as well as other characteristics such as buildings, parks, and waterways.

The trimming of the downloaded OSM map to match the actual geographical boundaries has been carried out with the help of tools called ***osmium*** [17] and ***osmfilter*** [18]. The ***osmfilter*** tool is used again to extract pertinent details from the map for the generation of scenarios. Of all the elements on the map, road networks are the most crucial to maintain. In OSM maps, roads, routes, or routes are labeled using a key element called 'highway.' Based on the type of road network, the highway key can take on a variety of values, such as'motorway', 'primary','secondary', and so on. For example, a road would be marked as 'highway=motorway' on the map. Depending on the needs of the scenario, other characteristics such as buildings, boundaries, and land can be retained in addition to road networks.

The above method is used to selectively retain the features of the Berlin map. For the current Berlin scenario, the roads that can be used by motor vehicles and the boundaries are retained. ***JOSM*** [19], which is an application that offers a visual interface to modify OSM files, can be used to make any necessary adjustments to the OSM file. The next step after the Berlin network is to identify the detector locations on the network. The detector locations where the traffic flow values recorded in [12] are expressed in the WGS84 coordinates. The location of the detectors in the actual data set can be visualized in Figure 2.

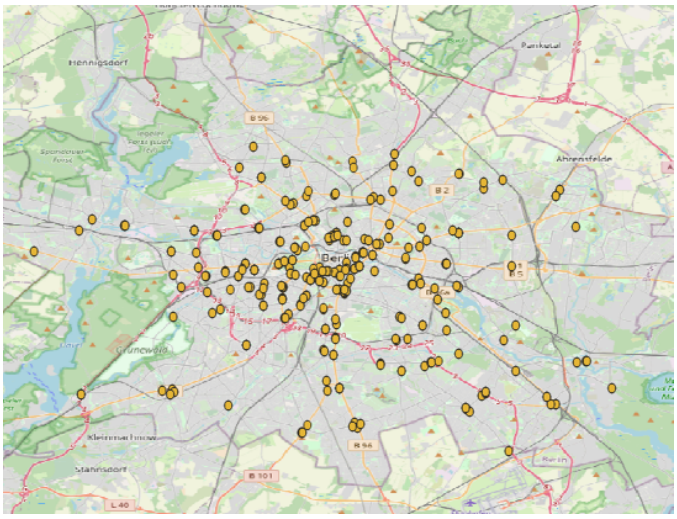


Fig. 2: Location of traffic detectors in Berlin dataset

C. Step3: Initialize the Scenario with Random Traffic Flows

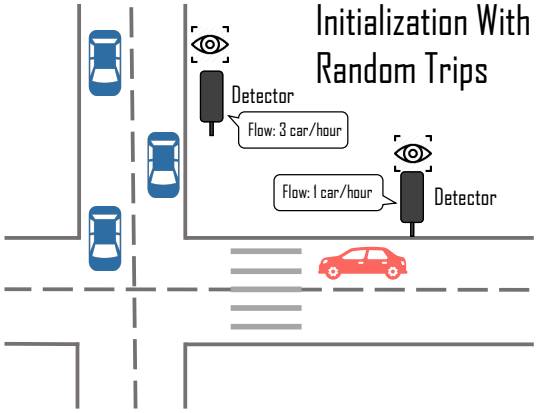
The primary objective of demand generation is to allocate routes to vehicles so that the estimated number of vehicles traveling on specific routes matches the actual traffic count data. As mentioned in the previous section, the ***routesampler.py tool*** in SUMO can perform this task by selecting routes based on input routes and traffic count data. To ensure that the output route file is reliable and precise, it is crucial to ensure that the provided sample routes closely resemble realistic routes taken in real world scenarios. Since the route sampler tool only takes route files as input and there is no direct way to generate routes in SUMO, a set of random trips are created by the ***randomTrips.py***. Trips are similar to the route file but do not specify the entire list of edges to be covered in the network file. Instead, it mostly has details about the start edge and the end edge. The created trips can eventually be fed to a tool called as ***duarouter*** which can create a route file when the route file is provided as input.

However, the routes generated by randomTrips.py usually cannot cover all the lanes in the scenarios, which will fail the calibration in the next step, since routeSampler.py cannot modify traffic flows with zero traffic information. Therefore, in addition to the random trips, we also generate Traffic Assignment Zone (TAZ) trips to cover all TAZ areas and lanes

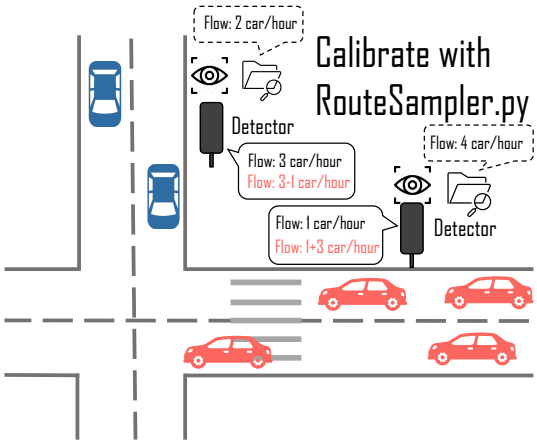
in the scenario.

D. Step4: Calibrate Traffic Flows as Predicted

Figure 3 shows how routeSampler calibrates traffic flows to match the predicted ones. The two generated route files (random trips and TAZ trips) are provided as input along with the edge data file that indicates the number of vehicle counts to be maintained for particular roads in the scenario. These vehicle counts are in accordance with the prediction made by the applied ML model. The routeSampler tool converts vehicle counts matching into a Linear Programming (LP) problem where a multiple set of routes is selected so that vehicle counts on the roads are matched as well as possible. With the optimize option set to full, no restrictions on the route distribution are set, and any route can be used as often as needed to reach the counts.



(a) Before calibration: the traffic flows are generated randomly



(b) After calibration: the traffic are matched with the predicted

Fig. 3: Illustration of how the routeSampler.py calibrates the traffic flows. (a) Before calibration (b) After calibration

V. Result

A. Results of different ML-models

Several candidate models are applied in the traffic flow dataset consisting of around 40 cities. The metric of Symmetric Mean Average Percentage Error (SMAPE), which is an accuracy measure based on relative errors, and Mean Average Error (MAE) are used to compare the performance of the different candidate models used. Table V indicates the relative performance of the various machine learning models applied.

MODEL	SMAPE%	MAE
Linear Regression Model	72.08	194.49
Linear SVM Model	71.53	187.16
Polynomial Regression Model	69.51	172.34
Decision Trees Model	19.74	79.85
Random Forest Model	17.12	71.04

TABLE V: Performance measure of applied machine learning models

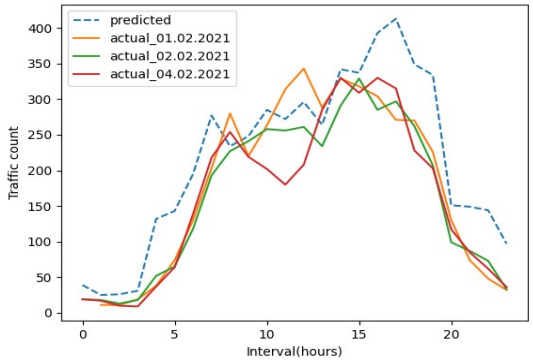
The Linear Regression and Linear SVM models have relatively high SMAPE values, indicating that their predictions are not very accurate. The Polynomial Regression Model has a slightly lower SMAPE value compared to the linear models, which means it may be a better predictor for this particular data set. A polynomial SVM model was not a viable option for the data set, as the training time is very high. The Decision Trees and Random Forest models have the lowest SMAPE values, which suggests that they are the most accurate predictors among the models that were evaluated. Overall, the Decision Trees and Random Forest models seem to be the best options to make predictions based on this data set, as they have the lowest SMAPE values. While Decision Trees are a type of non-parametric supervised learning algorithm that predicts the value of a target variable by learning simple decision rules from the data features, Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions to obtain a more accurate and stable model. Reduce overfitting by building multiple decision trees with different subsets of the training data and features. By combining the predictions of these trees, the model can capture more of the underlying patterns in the data and produce more accurate predictions on new data, as seen by the better SMAPE value in the table. Random forests are also more robust to outliers.

B. Results of Predicted vs. Actual Traffic Flows

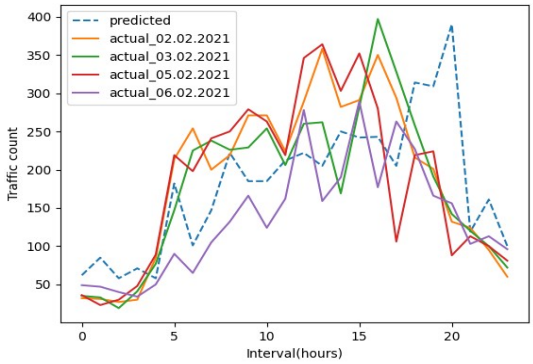
The aim of this work is to generate a scenario that is representative of traffic conditions in Berlin with minimal input data. Comparison of the predicted traffic flow values with actual traffic flow values also gives an idea of the performance. For the city of Berlin, actual traffic counts are available from 2015 at [12]. The data are comprehensive, as they provide hourly traffic data for various detectors not only for different years, but also for different days of every month. As expected, the actual traffic data recorded by the detectors vary according to different dates. However, by comparison of the predicted data and the actual detector data values, it is possible to assess the degree of correlation between them. The machine learning prediction model's estimated traffic values are graphed alongside the actual traffic values, which are available hourly for multiple days.

To allow for a direct comparison, the prediction was generated to reflect traffic flows detected in 1-hour intervals over

a 24-hour period. Figure 4 below shows the results of two among many detectors in the Berlin scenario, the observation interval of the vehicle count is from 0:00 to 23:59 o'clock, and different weekdays in Berlin are observed. The colored solid lines show the traffic count over time for dates from 02.02.2021 to 06.02.2021 and the dashed blue line shows the predicted traffic count based the ML-model trained by the UTD-19 dataset, note that only one single day is predicted. As a result, the general traffic pattern is captured by the trained ML-model over the 24-hour observation interval in Berlin.



(a) Detector1: located near the highway



(b) Detector2: located near the junction in city

Fig. 4: Result of the predicted vs. actual traffic flows; The colored solid lines represent actual traffic flows measured on different weekdays from the test Berlin dataset; The dashed-blue line represents the predicted traffic flow on single day trained with UTD-19 dataset (a) Detector 1, located near the highway (b) Detector2, located near the junction in city

For the result of the detector located near the highway, the traffic pattern shows a less variation from different dates and captures the typical peak-hour and off-peak-hour patterns for the traffic, where the predicted values mirror the trend of increasing traffic during morning hours and decreasing traffic during later parts of the day. The plot reveals a remarkable feature of a traffic peak that occurs around 7 hours, consistent with the highest value observed in the actual traffic data. Despite the variation between predicted and actual values between the 15 to 20 o'clock time period, the ascending and descending traffic patterns in this time frame still align with the actual traffic data pattern.

However, as for the result of the detector located near the city junction, the traffic pattern shows a highly dynamic and fast changing trend, where the traffic count differs largely according to the date time measured. However, the general peak and off-peak hour pattern is still captured by the ML-model and the model can adapt itself to the quickly changing traffic counts.

C. Results of Total Traffic Flow Generation

After the predicted traffic flow values are obtained from the ML-based model, the generation of edge-based count values must be satisfied to align with the predicted traffic flow values. This is followed by the route generation step, where the RouteSampler tool is used to match the edge-based count values. To evaluate the performance of matching count traffic values, the Geoffrey E Havers (GEH) value criterion named after its inventor Geoffrey E. Havers is introduced as one of several statistical measures that can be used to compare two sets of traffic volumes. The GEH is defined by the equation below:

$$GEH = \sqrt{\frac{2(M-C)^2}{M+C}} \tag{1}$$

where,

- C = traffic count predicted
- M = traffic count observed after route generation

As a result, a GEH value of less than 5 for 85 percent of all detectors after running the RouteSampler, and thus can be considered as a good acceptability guideline as mentioned in [20].

Figure 5 represents the trajectory of the total number of vehicles running in the Berlin scenario for 24 hours. The solid line represents the actual traffic inside the Berlin scenario, while the dashed represents the matched traffic flows by the RouteSampler based on prediction of the ML-model. From the result, a delay or time shift between the actual and predicted traffic flows is observed, and the reason is due to the RouteSampler. During matching the traffic count value to the detector provided by the prediction, the RouteSampler inserts and deletes the number of vehicles. However, to reveal the traffic pattern in the real world, vehicles need to be removed from the street after they have completed their routes rather than be deleted immediately, which results in a time shift. However, the general traffic pattern is captured by the ML-model, where the peak and off-peak traffic patterns are revealed.

VI. Conclusion

A. Conclusion

This work introduces an advanced approach to generate large-scale traffic demands for SUMO simulations using ML-based predictions. As a result, a Berlin scenario with an area of 891,7 km^2, over 260 traffic detectors and more than 1.2 million vehicles are generated for a 24-hour simulation. Additionally, a machine learning-based prediction model is

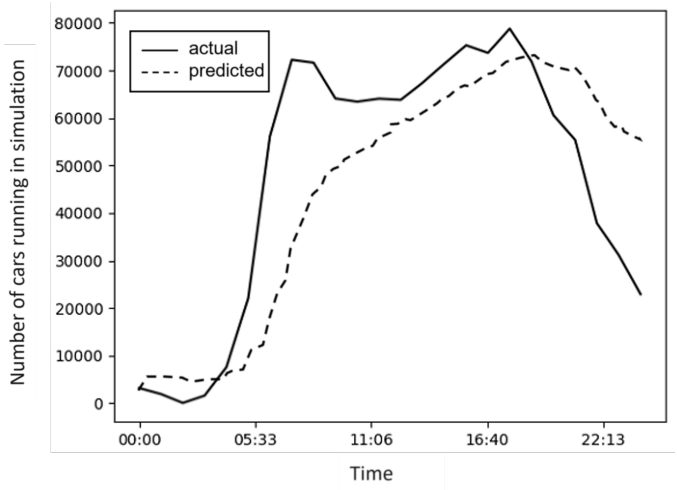


Fig. 5: Result of the total number of vehicles in the Berlin scenario over 24 hours; The solid line is the actual value within the whole area of Berlin; The dashed line is the matched with the RouteSampler based on the predicted traffic flows

used to make traffic flow predictions, using a large and representative data set. The prediction task employs a random forest model, chosen based on the metric of the lowest symmetric average percentage error. A comparison between predicted traffic flow values and actual traffic flow values obtained externally demonstrates that predicted traffic flow values exhibit patterns similar to actual values over a 24-hour period. Following the traffic flow prediction step, route files are generated and temporally distributed to match the predicted traffic value counts throughout the 24-hour duration, with the help of relevant transformations. In the final simulation, the trajectory of the total number of vehicles in use is observed to be similar to the predicted values.

B. Future Scope

As a potential future development, the use of more sophisticated Machine Learning models such as Artificial Neural Networks could be explored to increase the accuracy of traffic flow prediction. Additionally, the route generation step could be optimized to reduce the number of vehicles that wait and are stuck in traffic. The existing dataset could be extended to accommodate new data and features, and further tests of the trained model could be conducted. Furthermore, in this work, the detectors were created using the Berlin test dataset, but the user can also choose their own detectors to obtain the predicted value. Therefore, it is possible to assess the effect of the number of detectors on prediction performance in the future.

Currently, the generation of routes is based on the assumption that all vehicles are alone on the network and take the fastest routes to reach the destination. Other traffic assignment methods, such as iterative assignment, that take into account travel times in a traffic-loaded network, can be incorporated to decrease the number of vehicles getting jammed.

ACKNOWLEDGMENT

This work has been supported by the German Federal Ministry for Education and Research (BMBF) as part of the ICASMobility project. The authors alone are responsible for the content of the paper.

REFERENCES

[1] L. Bedogni, M. Gramaglia, A. Vesco, M. Fiore, J. Härri, and F. Ferrero, "The bologna ringway dataset: Improving road network conversion in sumo and validating urban mobility via navigation services," *IEEE Transactions on Vehicular Technology*, vol. 64, no. 12, pp. 5464–5476, 2015.

[2] K.-H. Kastner, R. Keber, P. Pau, and M. Samal, "Real-time traffic conditions with sumo for its austria west," in *Simulation of Urban Mobility: First International Conference, SUMO 2013, Berlin, Germany, May 15-17, 2013. Revised Selected Papers 1*. Springer, 2014, pp. 146–159.

[3] M. Rapelli, C. Casetti, and G. Gagliardi, "Tust: From raw data to vehicular traffic simulation in turin," in *2019 IEEE/ACM 23rd International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*. IEEE, 2019, pp. 1–8.

[4] S. Uppoor, O. Trullols-Cruces, M. Fiore, and J. M. Barcelo-Ordinas, "Generation and analysis of a large-scale urban vehicular mobility dataset," *IEEE Transactions on Mobile Computing*, vol. 13, no. 5, pp. 1061–1075, 2014.

[5] L. Codeca and J. Härri, "Monaco sumo traffic (most) scenario: A 3d mobility scenario for cooperative its," *EPiC Series in Engineering*, vol. 2, pp. 43–55, 2018.

[6] L. Codecá, R. Frank, S. Faye, and T. Engel, "Luxembourg sumo traffic (lust) scenario: Traffic demand evaluation," *IEEE Intelligent Transportation Systems Magazine*, vol. 9, no. 2, pp. 52–63, 2017.

[7] S. Partani, A. Qiu, R. Sattiraju, S. Tayade, and H. D. Schotten, "Quantitative assessment of ccam applications on greenhouse gas emissions," in *2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring)*, 2022, pp. 1–6.

[8] A. Qiu, S. Partani, D. Wang, and H. D. Schotten, "Quantitative assessment of penetration rates of ccam applications on ghg emissions in eu27," in *2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring)*, 2023, pp. 1–7.

[9] Y. Pigné, G. Danoy, and P. Bouvry, "A vehicular mobility model based on real traffic counting data." *Nets4Cars/Nets4Trains*, vol. 6596, pp. 131–142, 2011.

[10] J. Soares, C. Lobo, Z. Vale, and P. De Moura Oliveira, "Realistic traffic scenarios using a census methodology: Vila real case study," in *2014 IEEE PES General Meeting| Conference & Exposition*. IEEE, 2014, pp. 1–5.

[11] A. Loder, L. Ambühl, M. Menendez, and K. W. Axhausen, "Understanding traffic capacity of urban networks," *Scientific reports*, vol. 9, no. 1, p. 16283, 2019.

[12] "digitale_platform_stadtverkehr," https://daten.berlin.de/datensaetze/verkehrsdetektion-berlin, accessed: 2023-03-15.

[13] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds. IOS Press, 2016, pp. 87 – 90.

[14] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 51 – 56.

[15] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.

[16] OpenStreetMap contributors, "Planet dump retrieved from https://planet.osm.org ," https://www.openstreetmap.org , 2017.

[17] "Osmium," https://osmcode.org/osmium-tool/, accessed: 2023-03-15.

[18] "Osmfilter," https://wiki.openstreetmap.org/wiki/Osmfilter, accessed: 2023-03-15.

[19] "Josm," https://josm.openstreetmap.de/, accessed: 2023-03-15.

[20] E. National Highways, *Design Manual for roads and bridges*, 1996.